

Radar — Informe v7

Capa de recall semántico para prevenir la duplicación de lógica en código generado por IA

1. Resumen ejecutivo

Los agentes de IA generan código sin ver toda la base sobre la que trabajan. El resultado medible es que **reescriben lógica que ya existe** en lugar de reutilizarla, inflando los repositorios de duplicación que después hay que mantener.

Radar entiende qué *hace* cada función de un repositorio —no cómo está escrita— y lo expone donde se toman las decisiones de código. Su forma más defendible no es auditar deuda existente, sino **prevenir la duplicación en el momento de generar**: cada vez que un agente está por escribir una función, consulta primero si esa lógica ya existe y la reutiliza en lugar de duplicarla.

Técnicamente, Radar es **una capa de recall determinística y barata, con un LLM opcional encima para el juicio fino** (arquitectura de embudo: retrieve-then-rerank). El lookup vectorial baja de millones de funciones a un puñado de candidatos en milisegundos y de forma reproducible; sobre ese puñado, opcionalmente, un modelo juzga si son duplicados reales. Esa misma infraestructura se expone en tres superficies sobre el mismo motor:

1. **Gate de CI determinístico** — una compuerta de merge reproducible que marca o bloquea cuando la similitud supera un umbral calibrado. Es el wedge comercial y de demo: binario, medible y reproducible.
2. **Servidor MCP** — la tool `find_similar_function`, que el agente consulta *antes* de generar, previniendo la duplicación en origen a escala y frecuencia donde un chequeo costoso por función sería impracticable.
3. **Auditoría** — un dashboard para explorar el mapa de duplicación del repo (onboarding y revisión). Secundario: es el tablero, no el producto.

El modelo de negocio es infraestructura B2B con product-led growth: gratis en GitHub para entrar, pago para repos privados y on-prem. El posicionamiento no es "somos más baratos" (el costo de inferencia es casi nulo y no sostiene una venta) sino "prevención determinística de duplicación, donde se decide".

2. El problema y el marco de innovación

La frontera de innovación actual en herramientas de desarrollo es la **inteligencia de código nativa de IA**: agentes que entienden el repositorio completo, asisten en la generación y participan del review. Esa categoría ya está validada —el mercado la reconoce como el futuro y hay capital y adopción detrás—. Radar parte de esa premisa: no discute que la IA deba entender y asistir el código; **se para sobre esa frontera y aporta una primitiva específica que falta dentro de ella** —el recall semántico determinístico para prevenir duplicación—.

El problema concreto: el *vibe coding* introduce una "deuda técnica semántica". Los agentes operan con ventanas de contexto limitadas y solo ven los archivos abiertos, así que ignoran las abstracciones que ya existen en el repo y reimplementan lógica en vez de reutilizarla.

La evidencia de mercado es sólida pero conviene leerla con precisión:

- Según GitClear (211 millones de líneas analizadas), los bloques con 5 o más líneas duplicadas se multiplicaron por **8** durante 2024, y el copy/paste superó por primera vez a las líneas "movidas" (señal de refactor), que cayeron por debajo del 10%.
- **Matiz importante:** ese 8x mide duplicación *sintáctica* dentro de un mismo commit (copy/paste literal). Es el problema fácil, el que el análisis estático ya detecta. Radar ataca el problema *más difícil*: la duplicación **semántica** —misma lógica, distinta sintaxis— que aparece cuando un agente reimplementa de cero algo que existía en otro lado del repo, y que las herramientas legacy no ven porque buscan dependencias estructurales, no equivalencia lógica.

Aviso de honestidad sobre la urgencia: la duplicación es **deuda de segundo orden** —molesta, encarece el mantenimiento, pero rara vez tira producción—. Se vuelve un dolor pagable sobre todo en monorepos enormes, organizaciones con muchos repos, y equipos que ya adoptaron agentes en masa. Fuera de esos perfiles es "lindo de tener", no urgente.

3. El producto

Radar es **un solo motor** expuesto en distintas superficies. El motor parte el código en funciones, las convierte en vectores que capturan su intención algorítmica y los indexa para buscar por similitud. El resultado es un mapa donde las funciones que hacen lo mismo quedan cerca, sin importar cómo estén escritas.

La idea central, en una frase: **Radar es la capa de recall; el LLM (propio o del cliente) es el rerank opcional**. El lookup vectorial baja de millones de funciones a un puñado de candidatos —barato, rápido, determinístico—; sobre ese puñado, un LLM puede juzgar si son duplicados reales. El recall preciso y barato hace que el paso caro sea chico u opcional.

Las cuatro superficies, ordenadas por valor estratégico:

1. **Gate de CI determinístico (el wedge más afilado).** Una compuerta de merge reproducible: si la similitud semántica de un cambio supera un umbral calibrado, se marca o se bloquea, **igual en cada corrida**. La reproducibilidad es el requisito de una compuerta de CI —pasa o no pasa, hoy igual que mañana—, y es justo lo que un chequeo estocástico no puede garantizar. Es el ancla comercial y de demo porque es binario, demostrable y medible.
2. **Servidor MCP — prevención in-loop (el núcleo de futuro).** El índice se expone vía Model Context Protocol con una tool tipo `find_similar_function(code)`. El agente la consulta *antes* de generar y reutiliza en vez de duplicar. Su justificación no es el costo por llamada, sino que a frecuencia de "por función" —miles de veces por sesión— un chequeo caro y bloqueante es impracticable y un lookup vectorial no.
3. **Visualizador de auditoría — el tablero, no el producto.** Grafo donde las lógicas idénticas colapsan en clústeres, para onboarding y revisión. Secundario: sin IA propia, no se vende solo.
4. **Generación de PRs de deduplicación — la acción.** Desde el mapa, el Tech Lead autoriza que Radar genere los PRs que unifican el código duplicado. Cierra el ciclo de detección a resolución.

Por qué el cliente elegiría Radar (y cuándo no)

Los tres argumentos no pesan igual; conviene no mezclarlos:

- **Costo (débil).** El costo absoluto de una llamada de búsqueda es trivial y cae en órdenes de magnitud por año. Nadie cambia de herramienta para ahorrar fracciones de centavo. **Como argumento de venta aislado, no alcanza.**
- **Latencia (condicional).** Solo importa si el chequeo es bloqueante e inline (en el editor, en un pre-commit, en el loop del agente). Si el punto de integración valioso es el PR —pocas veces por día—, la latencia de un chequeo más pesado es invisible. Toda la propuesta depende de que exista demanda real de prevención más fina que el PR; esa demanda es plausible pero **aún no está probada**.
- **Determinismo (sólido pero angosto).** Es el único que se sostiene solo, y no por plata: no se puede poner un gate de merge bloqueante sobre un chequeo estocástico. El reverso honesto: el determinismo viene de ser **más tonto** —un umbral fijo sobre vectores dispara falsos positivos y pierde duplicados que un modelo razonando sí distinguiría—. Por eso gana para *bloquear* (reproducibilidad > inteligencia) y conviene complementarlo para *guiar* (inteligencia > reproducibilidad). De ahí el embudo: recall determinístico + rerank LLM.

4. Modelo de negocios

En una frase: infraestructura B2B por suscripción a equipos de desarrollo, con crecimiento por producto (PLG) —entrás gratis, te volvéis indispensable, cobráis por equipo.

Perfil del cliente

- **Usuario que lo prueba:** el desarrollador individual, vía plan gratis. No firma cheques; genera adopción orgánica y datos.
- **Comprador económico:** el liderazgo de ingeniería (Eng Manager / Staff / Tech Lead / VP Eng) en equipos donde la duplicación duele de verdad: **monorepos grandes** sin mapa completo, **organizaciones multi-repo** que reimplementan lógica entre equipos, y **adoptantes masivos de agentes** que ven crecer el código sin control.
- **A quién NO le vendés todavía:** al equipo chico con un repo que entra en la ventana de contexto de un agente. Ahí "uso el agente y listo" tiene razón.

Cómo se vende (escalera de monetización)

- **Free (GitHub Marketplace):** check de CI gratis para repos públicos/chicos. No monetiza: gana visibilidad (adopción dev-led) y señal de uso agregada.
- **Team (corazón del ingreso):** por-seat o por-repo. Repos privados, mapa org-wide, gate determinístico y PRs de deduplicación.
- **Enterprise:** on-prem (el código no sale), SSO y servidor MCP org-wide.
- **Capa MCP:** se ofrece como expansión, **no** como monetización principal —cobrar por llamada desincentivaría el uso del agente que se quiere fomentar—.

Defensa (foso)

Al simplificar el producto a la capa de recall sin validación humana, el foso por datos etiquetados desaparece —era una pata débil de todos modos—. Lo que queda es más modesto y honesto: Radar vive en el paso de *retrieval* determinístico, donde la mayoría de las herramientas de inteligencia de código no compiten (viven en el *razonamiento*: un modelo que interpreta y comenta). La defensa real a corto plazo es **llegar primero con el wedge más afilado** (el gate determinístico y la prevención in-loop) y el **índice afinado a las convenciones de cada cliente**, que genera switching cost una vez integrado en el CI. No es un foso fuerte: es una ventaja de foco y de tiempo.

5. Arquitectura técnica (resumen)

Pipeline basado en un encoder especializado y barato —**no un LLM generativo en el camino crítico**—, reproducible, on-prem capable y de bajísimo costo de inferencia:

- **Extracción:** Tree-sitter delimita y extrae las funciones (chunking, no alimentar ASTs al encoder).
- **Normalización:** se canonizan nombres de variables y se quitan comentarios/whitespace para colapsar ruido sintáctico antes de embeber.
- **Embeddings (recall):** un encoder de código vigente que capture la intención algorítmica. **Nota:** `jina-embeddings-v2-base-code` (la elección original) figura como deprecado por Jina; usar la generación actual (p. ej. `jina-embeddings-v4`, que incorpora code retrieval) u otro code-embedder vigente.

- **Índice y búsqueda (recall determinístico):** FAISS (kNN con umbral de confianza calibrable). Devuelve el mismo resultado en cada corrida → habilita el gate de CI y la consulta in-loop.
- **Rerank opcional (precisión):** sobre los pocos candidatos del recall, un LLM puede juzgar si son duplicados reales. Es el paso "inteligente" del embudo, chico porque opera sobre 3–5 candidatos, no sobre todo el repo. Opcional: el cliente puede usar su propio modelo.
- **Visualización (secundaria):** UMAP + HDBSCAN, render con react-force-graph sobre WebGL.

El argumento técnico de venta es la **precisión del recall**: precision/recall sobre un set held-out de duplicados conocidos, con sensibilidad ajustable porque **el falso positivo es el riesgo número uno** (un gate que bloquea un merge legítimo se desactiva en una semana).

6. Diseño del MCP: contrato y flujo de datos

El producto, simplificado, es **un servidor MCP que el agente consulta en tiempo de generación** para saber si la función que está por escribir ya existe. Sin humano en el loop: el agente pregunta, Radar responde con un veredicto determinístico contra un umbral calibrado offline, y el agente decide solo si reutiliza o escribe. Todo lo que sigue es una sola llamada sincrónica; no hay revisión humana, ni cola asíncrona, ni etiquetado.

6.1 Contrato de la tool

Una sola tool, `find_similar_function`. Regla de diseño: la respuesta es **accionable, no solo informativa** (devuelve cómo reutilizar, no solo que hay un duplicado), y el recall es **determinístico** —el resumen de cada función se precomputa al indexar, sin LLM en el camino crítico—.

Input:

```
find_similar_function({
    code: string,      // requerido: la función que el agente está por escribir

    language?: string, // opcional, default inferido ("python")

    intent?: string,   // opcional: NL de qué intenta hacer (mejora recall, cuesta tokens)

    top_k?: int        // opcional, default 3
})
```

El umbral NO es parámetro del agente; es config server-side (calibrada offline). El agente no debería poder aflojarlo.

Output:

```
{  
  
  query_id: string,           // id del evento (telemetría)  
  
  verdict: "duplicate" | "similar" | "novel", // contra el umbral calibrado  
  
  matches: [  
  
    {  
  
      match_id: string,      // id estable del par consulta↔candidato (telemetría/caché)  
  
      name: string,          // "parse_dates"  
  
      signature: string,      // "def parse_dates(s: str, tz: str = 'UTC') -> datetime"  
  
      location: string,       // "utils/dates.py:42-58"  
  
      summary: string,        // 1 línea precomputada: "parsea fechas ISO a datetime con tz"  
  
      import_statement: string, // "from utils.dates import parse_dates" ← lo accionable  
  
      similarity: float       // 0.0–1.0  
  
    }  
  
  ]  
  
}
```

El **verdict** lo decide el servidor de forma determinística contra el umbral (corte alto → **duplicate**, banda media → **similar**, por debajo → **novel**). El agente recibe hechos + veredicto y **decide solo** si reutiliza (es el cerebro; la tool es el sensor). El **import_statement** es lo que vuelve trivial reutilizar en vez de regenerar.

6.2 Calibración del umbral (sin humano)

El umbral se fija **offline**, midiendo precision/recall sobre un set held-out de duplicados conocidos, y se expone como config ajustable por repo/cliente. No se aprende de clicks humanos: se setea una vez, se versiona, y se reajusta cuando se mide drift. Como **el falso positivo es el riesgo número uno** (un agente que reutiliza algo que no correspondía introduce un bug sutil), la calibración prioriza **precisión sobre recall** —preferimos dejar pasar algún duplicado antes que marcar uno falso—.

6.3 Flujo de datos

Un único camino, sincrónico:

Agente (Claude Code) --code--> [Radar MCP]

normaliza → embed → FAISS kNN → umbral → (rerank opcional)
| (lee Índice FAISS, construido offline)
|
v

Agente <--matches + verdict + import_statement-- [Radar MCP]

→ el agente reutiliza o escribe nuevo

El mismo motor de recall + umbral puede exponerse también como **gate de CI** (check determinístico pasa/no-pasa sobre el diff del PR), sin humano: es la misma infraestructura, otra puerta. No son dos motores: es uno expuesto dos veces (agente vía MCP, gate de CI), lo que mantiene el sistema barato y coherente.

El índice FAISS y el metadata store se construyen **offline** por el pipeline de indexado (tree-sitter → normalización → embeddings → FAISS) y el motor los carga al arrancar. La consulta online no reindexar nada: solo embebe la función entrante y busca.

7. Riesgos y contexto competitivo

El espacio de la inteligencia de código nativa de IA está activo y bien financiado (Greptile, Qodo y otros construyen review asistido, grafos de repo y motores de contexto vía MCP). Eso es, ante todo, **validación de mercado**: confirma que la categoría en la que se inserta Radar es real y que el comprador ya está educado. La diferenciación de Radar dentro de esa categoría es estrecha pero concreta —el recall determinístico para prevención de duplicación— y conviene tratarla como un wedge para entrar, no como un foso permanente.

Los riesgos genuinos, en orden de importancia:

1. **Falsos positivos (riesgo de producto N.º 1).** El mercado castiga a las herramientas ruidosas; un gate que bloquea un merge legítimo se desactiva enseguida. Mitigación: umbral calibrado offline, optimizar precisión sobre recall (mejor dejar pasar un duplicado que marcar uno falso).
2. **La diferenciación es angosta y absorbible.** El gate determinístico de duplicación es valioso pero un competidor de plataforma podría sumarlo como una feature. La ventaja es de velocidad-al-wedge y foco, no estructural a largo plazo.

3. **Demanda no probada para la prevención más fina que el PR.** Si el mercado solo se preocupa por la duplicación en el review, el caso in-loop pierde fuerza. Es una apuesta a la dirección agéntica, razonable pero no garantizada.
 4. **Duplicación = deuda de segundo orden.** Urgente solo en casos extremos (monorepos enormes, costos de mantenimiento medibles), justo los difíciles de mostrar en una demo.
 5. **TAM angosto como producto mono-métrica.** El foco da claridad pero limita el techo; el camino de crecimiento es de wedge a plataforma, no quedarse en la feature.
 6. **La tecnología no es novedosa en sí.** La detección de clones semánticos es campo académico maduro; la novedad está en la productización (prevención determinística vía MCP), no en el algoritmo. No hay secreto que proteja.
 7. **El propio agente aproxima la función.** Para repos que entran en contexto, un agente detecta duplicación y aproxima la prevención. La ventaja de Radar es operativa (costo/latencia/cobertura/determinismo a escala), no de capacidad.
 8. **Herramientas legacy ya instaladas (SonarQube, Codacy).** Detectan duplicación sintáctica y están en miles de pipelines; el cliente puede sentir que "ya tiene una herramienta de calidad".
-

8. Viabilidad para la hackathon

Veredicto: hay material de sobra para un MVP ganador. La regla es una: **elegir una columna vertebral y matar el resto para el demo.**

El MVP a construir

La superficie que cumple las dos consignas (tiene IA y es vendible) y produce el mejor "ah, mirá" es el **gate determinístico + la prevención vía MCP**. El demo es un antes/después:

1. Indexar una vez un repo Python chico con duplicados sembrados a propósito: tree-sitter → normalización → encoder → FAISS.
2. Exponer **una sola** tool MCP: `find_similar_function`.
3. **Antes:** pedirle a un agente "escribime una función que haga X" (donde X ya existe). Sin la tool, duplica. Mostrarlo.
4. **Después:** activar el MCP. Mismo pedido. El agente consulta `find_similar_function`, encuentra la función existente y la reutiliza.
5. **Cierre:** mostrar el gate determinístico — el mismo cambio, corrido dos veces, da el mismo resultado reproducible.

Ese contraste es el "ah, mirá", muestra IA (un agente razonando sobre el retrieval) y prueba el pitch en 30 segundos, esquivando el falso positivo porque es un caso curado.

Qué afilar

- **Un número propio del demo:** "el agente generó 3 implementaciones duplicadas de lógica existente; con Radar, 0". Vale más que el 8x ajeno porque es tuyo y en vivo.

- **El gate determinístico como ancla**, no el costo. Mostrar reproducibilidad, no precio.
- **El argumento de escala/latencia como visión**, con el costo proyectado de un escaneo por función vs. el lookup (anticipando que brilla en repos grandes).
- **Encoder vigente**: no usar el modelo deprecado.
- **Frase preventiva sobre falsos positivos** metida antes de que la pregunten.
- **La categoría como respaldo, no como amenaza**: el capital invertido en inteligencia de código valida el mercado en el que entrás.

El reencuadre de venta

No salir con "detectamos clones semánticos" ni con "somos más baratos". Salir con:

"evitamos que tus agentes de IA reescriban código que ya tenés, con una compuerta determinística sobre la duplicación — menos código que mantener, menos slop". El demo lo prueba.

VENTA CON PERSPECTIVA A FUTURO. POR TRES MOTIVOS:

1. Los tokens van a ser más caros
2. Va a haber más vibe coding por lo que los riesgos y los costos de código duplicado aumentará.
3. RETROALIMENTATIVO. Más vibe coding -> más código duplicado -> peor vibe coding (y más caro)

CONSTRUCCIÓN PARA HACKATHON

Partes a implementar

A. Pipeline de indexado (offline, se corre una vez sobre el repo sembrado)

1. **Extractor de funciones** (tree-sitter, solo Python) → lista de funciones con metadata: nombre, archivo, rango de líneas, fuente.
2. **Normalizador** — canoniza nombres de variables, saca comentarios/whitespace.
3. **Embedder** — un code-encoder vigente (NO el jina v2-base-code deprecado; usá uno actual y chico para correr local sin drama de GPU) → vectores.
4. **Precómputo de `summary` e `import_statement`** por función. Para el MVP el `summary` puede ser la docstring o la firma; no necesitás un LLM acá.
5. **Constructor del índice** — FAISS + un metadata store (sqlite o un JSON) que mapea `vector_id` → `metadata + import_statement`.

B. Motor de consulta (online, el corazón = el contrato) 6.

`find_similar_function(code, ...)`: normaliza → embebe → kNN en FAISS → aplica umbral → arma `matches` (con signature, location, summary, import, match_id, similarity) → `verdict`. Determinístico. 7. **Umbral** — config server-side (para el MVP, un solo valor calibrado a mano). 8. *(Opcional, solo si sobra)* rerank LLM del candidato top.

C. Superficies (adaptadores delgados sobre B) 9. **CLI** — para tu desarrollo y para tirar el número de precisión. 10. **Servidor MCP stdio** exponiendo `find_similar_function`. 11. **Setup del agente** — Claude Code/Cursor apuntando al MCP local.

D. Assets del demo (lo más subestimado — separa tiempo) 12. **Repo sembrado** — un repo Python con duplicados plantados a propósito (la función X que "ya existe"). 13. **Set held-out** de pares duplicados conocidos para reportar precisión/recall (tu número propio). 14. **Guion antes/después** — el prompt que dispara duplicación sin la tool y reutilización con ella; y la "cereza del determinismo": correr el mismo chequeo dos veces y mostrar el mismo pasa/no-pasa.

E. Cortado del MVP (solo si sobra tiempo; se cuenta en el pitch, no se construye)

- Path asíncrono del HITL completo (sampler, ground-truth store, calibración), dashboard de accept/reject, Action de CI real, grafo 3D, fine-tuning, telemetría.
- Concesión mínima útil: un CLI que devuelve pasa/no-pasa demuestra el gate determinístico sin CI real, y un stub que graba accept/reject a un JSON vuelve tangible el flywheel sin construir la UI.

Cómo se coordinan

El **pipeline A** produce dos artefactos en disco: el índice FAISS y el metadata store. El **motor B** los carga al arrancar y sirve consultas. El **CLI (9)** y el **MCP (10)** son dos puertas de entrada finas que llaman al *mismo* motor B —no son dos implementaciones—. El **agente (11)** se conecta al MCP. El **script de eval (13)** llama a B directamente para sacar el número. Esto es justo el principio del informe: un solo motor expuesto varias veces.

Del flujo de datos que diagramamos, para el MVP construí **solo el camino sincrónico** (agente → MCP → recall → respuesta). El camino asíncrono del humano queda descripto en el pitch.

Orden de construcción (de-risqueando el demo)

Principio: las dos cosas que más probablemente explotan a último momento son la **conexión MCP-cliente** y la **calidad del recall** (falsos positivos). Ataca ambas temprano.

- **Bloque 0 — Setup (~1h)**. Esqueleto del repo, dependencias, firma de `find_similar_function` fijada como contrato. Confirmá que el encoder carga y embebe un string. En paralelo, alguien arma el repo sembrado (12).
- **Bloque 1 — Motor (lo más grande, ~medio día)**. Extracción → normalización → embeber todo → índice FAISS + metadata. Después la función de consulta. Testeá con un script plano. **Acá se gana o se pierde la precisión** — dejá tiempo para calibrar el umbral contra el set held-out.
- **Bloque 2 — Smoke test MCP (temprano, en paralelo, ~1-2h)**. Levantá un MCP stdio mínimo con una tool trivial y conectá Claude Code/Cursor. Confirmá que el agente la llama *antes* de que el motor esté listo. No lo dejes para el final.
- **Bloque 3 — Cablear motor real al MCP (~1-2h)**. Reemplazás la tool dummy por `find_similar_function`. El agente ya consulta el índice real.

- **Bloque 4 — El demo (~2-3h).** Afiná el antes/después hasta que el contraste sea limpio. Armá la cereza del determinismo (mismo chequeo ×2 → idéntico). Computá el precision/recall.
- **Bloque 5 — Pitch y red de seguridad.** Frases de una línea, guion de 3 min, y **grabá un video de respaldo del demo funcionando** antes de subir al escenario (los demos en vivo fallan).

DIVISION PARA DESARROLLO

H0 – H0:45 · Arranque conjunto (los 4). Fijen el contrato y, crítico, el **schema del function record** (la costura $P1 \leftrightarrow P2$) y el **encoder** (que corra en laptop, sin GPU). Cada uno clona, instala, confirma que importa sus librerías. No pasen de aquí sin que el encoder embeba un string y tree-sitter parsee un archivo —si algo de eso falla, lo descubren ahora y no a la hora 6—.

H0:45 – H4 · Construcción en paralelo, todos contra stubs.

- **P1 (núcleo):** expone `find_similar_function` como librería importable ya, devolviendo un match hardcodeado. Eso desbloquea a P3 y P4 en el minuto uno. Recién después construye lo real: embeber → FAISS → kNN → umbral → armar matches.
- **P2 (pipeline):** tree-sitter → normalizador → records → metadata store. Le entrega los records a P1 apenas los tenga (~H2-2:30). Ese handoff es el evento más importante de la mañana.
- **P3 (MCP):** levanta el MCP stdio con la tool dummy y **conecta Claude Code/Cursor antes de la H2**. Confirmá que el agente la llama. Esto es lo que más probablemente explota; si falla, querés saberlo temprano.
- **P4 (demo):** repo sembrado + set held-out, ambos sin depender de nadie. Para H3 los duplicados plantados están listos.

H4 · Punto de integración duro (los 4, ~30 min, inamovible). Motor real de P1 + records reales de P2 + MCP de P3 cableado al motor real. Acá tiene que verse al agente consultando el índice real **una vez, de punta a punta**, aunque feo. Si a la H4 esto no pasa, es la señal para recortar (ver abajo).

H4:30 – H6 · Calibración y demo. P1 + P4 calibran el umbral contra el set held-out —este es tu seguro contra el falso positivo, el riesgo número uno, y no es opcional—. P3 + P4 afinan el antes/después hasta que el contraste sea limpio y arman la cereza del determinismo (mismo chequeo ×2 → idéntico).

H6 – H7 · Congelamiento y red de seguridad. Code freeze. Ensayo completo del antes/después con los 4 mirando, y P4 **graba el video de respaldo**. Después de la H7 no se toca código salvo para apagar incendios.

H7 – H8 · Pitch. Guion de 3 minutos, frases de una línea, el número propio ("3 duplicados → 0"). P4 lidera, todos lo escuchan una vez